

カメラ操作と ユーザーインタラクション

ユーザーの操作で視点を動かし、対象を選ぶシーンをつくる

Week 5 2 hours Three.js × WebGL 実践講座

本日のアジェンダ

01

視点をコントロールする

OrbitControls で回す・寄る・パンする

30 min

02

対象を選択する

Raycaster でクリック・ホバーを判定する

40 min

03

ハンズオン演習

立方体をクリックで強調表示する

50 min

解説 60 min → ハンズオン 50 min → 振り返り 10 min

前提知識・準備物

第4回の到達点

- 画像テクスチャを map で貼れる
- normal / roughness で質感を出せる
- CubeTexture で背景を表現できる
- animate() で描画ループが回っている

今回必要なもの

前回のプロジェクト

my-threejs-app をそのまま使用

three/examples

OrbitControls を import できる構成

複数のオブジェクト

選択対象になる立方体を数個

インタラクションの3ステップ

Controls

操作

OrbitControls でカメラを動かす。回転・ズーム・パンを一括で提供

Raycaster

選択

画面の座標から光線を飛ばし、交差した対象を調べる

Events

反応

pointer イベントで操作を受け取り、状態を更新する

動かす → 調べる → 反応する の流れで、触れるシーンが生まれる

OrbitControls — 視点を操作する

マウスやタッチでカメラを回転・ズーム・パンできるようにする

```
import { OrbitControls } from
  'three/addons/controls/OrbitControls.js';
const controls =
  new OrbitControls(camera, renderer.domElement);
controls.enableDamping = true; // 慣性
// animate 内で controls.update();
```

enableDamping

慣性でなめらかに停止する

target

注視点（回転の中心）を指定

min/maxDistance

ズームの範囲を制限する

Raycaster — 対象を選択する

画面上の座標から光線を飛ばし、交差したオブジェクトを取得する

```
const raycaster = new THREE.Raycaster();  
const pointer = new THREE.Vector2();  
raycaster.setFromCamera(pointer, camera);  
const hits =  
  raycaster.intersectObjects(scene.children);  
const target = hits[0]?.object; // 最前面
```

setFromCamera

座標とカメラから光線を作る

intersectObjects

交差を近い順の配列で返す

hits[0].object

最も手前の対象を取り出す

Pointer Events — 操作を受け取る

マウスとタッチを同じ pointer イベントで扱い、座標を NDC に変換する

```
const el = renderer.domElement;  
el.addEventListener('pointermove', (e) => {  
  const r = el.getBoundingClientRect();  
  pointer.x = ((e.clientX-r.left)/r.width)*2-1;  
  pointer.y = -((e.clientY-r.top)/r.height)*2+1;  
});
```

pointermove

マウス・タッチ・ペンを統一

clientX/Y → NDC

画面座標を -1～1 に変換

touch-action

CSS でスクロール干渉を防ぐ

PART 02

カメラを使い分ける

シーンに合ったカメラと、座標変換のしくみを理解する

インタラクション = Controls × Raycaster × Events

Controls

視点

カメラの動きを操作する

×

Raycaster

選択

光線で対象を特定する

×

Events

入力

ポインタ操作を受け取る

3つがそろって初めて「触れるシーン」になる

Perspective と Orthographic

PerspectiveCamera

遠近感のある投影

用途 3D空間・自然な奥行き表現

引数 fov, aspect, near, far

特徴 遠くのものほど小さく見える

OrthographicCamera

平行投影

用途 UI・図面・アイソメ表現

引数 left, right, top, bottom, near, far

特徴 距離が変わっても大きさは一定

空間表現は Perspective、寸法を保つ図は Orthographic

座標変換：画面 → NDC

クリック位置を Three.js が扱う正規化デバイス座標(NDC)に変換する

画面座標 (px)

`clientX, clientY`

マウス・タッチの生の位置



正規化 (0～1)

`/ rect.width, height`

canvas の大きさに割る



NDC (-1～1)

`×2-1`、Yは反転

Three.js が使う座標系へ

```
pointer.x = (x / width) * 2 - 1;    // -1 ~ 1
pointer.y = -(y / height) * 2 + 1;  // Y軸は反転
```

最小コードでインタラクション

```
import * as THREE from 'three';
import { OrbitControls } from 'three/addons/controls/OrbitControls.js';
const controls = new OrbitControls(camera, renderer.domElement);
controls.enableDamping = true;
const raycaster = new THREE.Raycaster();
const pointer = new THREE.Vector2();
addEventListener('pointerdown', (e) => {
  pointer.set((e.clientX/innerWidth)*2-1, -(e.clientY/innerHeight)*2+1);
  raycaster.setFromCamera(pointer, camera);
  const hit = raycaster.intersectObjects(scene.children)[0];
  if (hit) hit.object.material.color.set('#22d3ee');
});
(function loop(){ controls.update(); renderer.render(scene,camera);
  requestAnimationFrame(loop); })();
```

インタラクションの流れ



毎フレームではなくイベント発生時に判定すると軽くなる

ホバーで対象を強調する

マウスが乗ったオブジェクトを記録し、発光色で反応させる

```
let hovered = null;  
const hit = raycaster.intersectObjects(targets)[0];  
const obj = hit ? hit.object : null;  
if (obj !== hovered) {  
  hovered?.material.emissive.set(0x000000);  
  obj?.material.emissive.set(0x224466);  
  hovered = obj;  
}
```

ホバーの記録

直前の対象を覚えて元に戻す

emissive で強調

発光色でハイライトする

なめらかに・軽く動かす

慣性・対象の限定・イベント頻度の調整で操作を快適にする

```
controls.enableDamping = true;    // 慣性を効かせる
controls.dampingFactor = 0.05;    // 効き具合
raycaster.layers.set(1);          // 判定対象を限定
// pointermove は throttle して負荷を下げる
```

カクつく・重いときは?

交差対象を絞り、pointermove を間引く。animate 内の controls.update() の呼び忘れにも注意する

よくあるつまずきポイント

Q

操作しても動かない

animate 内の controls.update() を確認する

Q

クリックが当たらない

NDC変換とYの符号、対象の指定を見直す

Q

選択位置がズれる

getBoundingClientRect で canvas 基準に直す

Q

スマホで反応しない

pointer イベントと touch-action を設定する

立方体をクリックで強調しよう

Step 1

立方体を並べる

数個の Mesh をシーンに配置

Step 2

OrbitControls

視点を回せるようにする

Step 3

Raycaster を用意

pointer 座標を NDC に変換

Step 4

クリック判定

intersectObjects で対象を取得

Step 5

強調表示

当たった立方体の色を変える

ゴール: 並んだ立方体から、クリックしたものが色を変えて反応する

まとめ & 次回予告

今日のまとめ

- OrbitControls で視点を操作する
- Raycaster で対象を選択する
- ポインタ座標を NDC に変換する
- ホバー・クリックで反応を付ける
- 交差対象を絞って軽く保つ

次回： SESSION 06

モデル読み込みと外部リソース

外部の3Dモデルを読み込み、シーンに組み込みます。

- glTF / GLB を GLTFLoader で読み込む
- モデルのスケール・中心を調整
- Raycaster と組み合わせて操作